Probabilistic Compute-in-Memory Design for Efficient Markov Chain Monte Carlo Sampling

Yihan Fu, Daijing Shi, Anjunyi Fan, *Student Member, IEEE*, Wenshuo Yue, Yuchao Yang, *Member, IEEE*, Ru Huang, *Fellow, IEEE*, and Bonan Yan, *Member, IEEE*

***Abstract*— Markov chain Monte Carlo (MCMC) is a widely used sampling method in modern artificial intelligence and probabilistic computing systems. It involves repetitive random number generations and thus often dominates the latency of probabilistic model computing. Hence, we propose a compute-in-memory (CIM) based MCMC design as a hardware acceleration solution. This work investigates SRAM bitcell stochasticity and proposes a novel "pseudo-read" operation, based on which we offer a block-wise random number generation circuit scheme for fast random number generation. Moreover, this work proposes a novel multi-stage exclusive-OR gate (MSXOR) design method to generate strictly uniformly distributed random numbers. The probability error deviating from a uniform distribution is suppressed under 10^{-6}. Also, this work presents a novel in-memory copy circuit scheme to realize data copy inside a CIM sub-array, significantly reducing the use of R/W circuits for power saving. Evaluated in a commercial 28-nm process development kit, this CIM-based MCMC design generates 4-bit∼32-bit samples with an energy efficiency of 0.53 pJ/sample and high throughput of up to 1066.7M samples/s. Compared to conventional processors, the overall energy efficiency improves 2.12×10^9 to 9.58×10^9 times.**

***Index Terms*— SRAM, processing-in-memory, Markov chain Monte Carlo sampling, MCMC accelerator, probabilistic programming, random number generator, arbitrary distribution.**

I. Introduction

MARKOV chain Monte Carlo (MCMC) method is an essential sampling technique used in statistics and artificial intelligence (AI) [1]. It generates samples that closely approximate a given multivariate distribution, i.e. prior distribution [2]. Unlike other typical sampling techniques such as rejection sampling, the MCMC method scales well with expanding state space and increasing dimensions. This algorithm provides a proper solution to those tasks featuring large datasets and high-dimensional models [3]. Specifically, MCMC sampling is widely used in probabilistic modeling [4], reinforcement learning [5], and scene understanding [6] (Fig. 1).

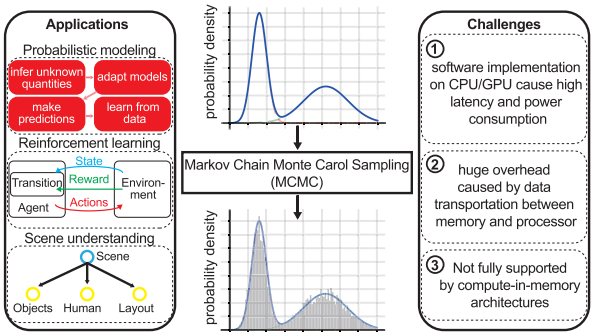


Fig. 1. Applications and challenges of MCMC computing.

However, the execution of MCMC algorithm on central processing units (CPUs) or graphic processing units (GPUs) only generates $10^{2\sim3}$ samples per second, which is far from the required sample generation speed for real-time probabilistic inference. For example, in the application of scene understanding [6], the system gets a parse graph as the optimal configuration of the 3D monitored scene at the beginning of one frame. The parse graph should be iteratively optimized by MCMC sampling with simulated annealing based on posterior probability before the end of this frame. But MCMC sampling based on GPU acceleration needs many seconds to generate enough samples for the optimization by frame, which apparently cannot meet the speed requirement, say, 33 ms/frame (30 frame per second), for real-time scene understanding in robots. The reason behind this is that every time a sample is generated in GPU, it needs to be transferred to the main memory. The frequent data transfer between processing units and memories causes long latency and high energy consumption, which dominates the overall computing process [7], [8], [9]. The acceleration of MCMC turns into a critical problem in the deployment of probabilistic machine

Manuscript received 14 July 2023; revised 1 October 2023 and 29 October 2023; accepted 21 November 2023. Date of publication 7 December 2023; date of current version 30 January 2024. This work was supported in part by the Natural Science Foundation of China under Grant 92264201, Grant 92364102, Grant T2350006, Grant 61925401, Grant 92064004, Grant 61927901, and Grant 92164302; and in part by 111 Project under Grant B18001. The work of Yuchao Yang was supported in part by the Fok Ying-Tong Education Foundation and in part by the Tencent Foundation through the Xplorer Prize. This article was recommended by Associate Editor X. Liu. *(Corresponding authors: Yuchao Yang; Ru Huang; Bonan Yan.)*

Yihan Fu, Daijing Shi, Anjunyi Fan, Wenshuo Yue, Ru Huang, and Bonan Yan are with the Institute for Artificial Intelligence and the Beijing Advanced Innovation Center for Integrated Circuits, School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: yihanfu@pku.edu.cn; daijingshi@pku.edu.cn; anjunyif@pku.edu.cn; yws2017@pku.edu.cn; ruhuang@pku.edu.cn; bonanyan@pku.edu.cn).

Yuchao Yang is with the Institute for Artificial Intelligence and the Beijing Advanced Innovation Center for Integrated Circuits, School of Integrated Circuits, Peking University, Beijing 100871, China, also with the School of Electronic and Computer Engineering, Peking University, Shenzhen 518055, China, and also with the Chinese Institute for Brain Research (CIBR), Beijing 102206, China (e-mail: yuchaoyang@pku.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSI.2023.3337529.

Digital Object Identifier 10.1109/TCSI.2023.3337529

© 2023 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

learning. This necessitates the invention of memory-centric hardware architectures to accelerate MCMC.

Prior works have explored MCMC sampling implementation on novel hardware platforms. Cai et al. [10] and Banerjee et al. [11] designed FPGA-based hardware accelerators that compromise sampling accuracy for gaining two to three orders of magnitude of performance-per-watt over CPU and GPU, but their work can only adapt MCMC sampling on Gaussian functions. Shukla et al. [12] invented MC^{2}RAM using SRAM bitcells, multipliers, and analog-to-digital converters (ADCs) to achieve fast Gaussian mixture models (GMMs) sampling; however, it failed to thoroughly investigate the random sample precision and the influence of non-ideal practical conditions, e.g. process variations and thermal fluctuation. Dalgaty et al. [13] exploited the variability of 16k memristor nanoscale devices to implement random number generation and demonstrated low-power memristor-based MCMC for an exemplary reinforcement learning application, but it remains a proof-of-concept work at device research level that relies on the software platform and C++ standard random math package for the accept/reject check.

This work proposes a compute-in-memory (CIM) design to address these challenges to construct a high-performance MCMC sampler accelerator. Featuring high sampling speed, expandable precision, and high parallelism, the proposed CIM-based MCMC design realizes a complete on-chip MCMC sampling iteration and has great potential as a standalone domain-specific core to be integrated into a system-on-a-chip (SoC). SRAM-based CIM technology has been considered to generate random samples, but it has been rarely considered to implement fully-fledged MCMC accelerator hardware. This work innovatively proposed a dedicated circuit for MCMC sampling based on an SRAM CIM architecture. It can realize MCMC sampling of arbitrary distribution.

The contribution of this work includes:

- We investigate the stochasticity of SRAM bitcell and propose a novel operating principle of the "pseudo-read" operation, based on which we offer a block-wise random number generation (RNG) circuit scheme for fast random number (sample) generating speed over 10^6 samples/s.
- We propose a novel multi-stage exclusive-OR gate (MSXOR) design method to correct a biased probability (e.g. ~40%) to generate strict 50% uniform distributed random numbers. The error probability achieves as low as 10^{-4}%. Further, we realize an efficient accept/reject check circuit module.
- We propose a novel in-memory copy circuit scheme to realize data copy between different bitcells, which can significantly reduce the use of R/W circuits for energy- and power-saving purposes.

To evaluate the proposed CIM design, we implement the proposed CIM-based MCMC design with a commercial process development kit (PDK) at 28 nm technology node with 6-transistor (6T) SRAM foundry bitcells. The proposed CIM-based MCMC design achieves high-accuracy sampling with an energy efficiency of 0.53 pJ/sample and high throughput of up to 1066.7M samples/s.

Algorithm 1 Metropolis-Hastings MCMC Sampling

Initialise $x^{(0)} \subset X$
For $\mathbf{i} = 0$ **to** $N-1$
Step 1: **Sample** $x^* \subset q(x^*|x^{(i)})$
Step 2: **Sample** $u \subset U[0,1]$
Step 3: **If** $u < \alpha(x^{(i)}, x^*) = min\{1, \frac{p(x^*)q(x^{(i)}|x^*)}{p(x^{(i)})q(x^*|x^{(i)})}\}$
$x^{(i+1)} = x^{(*)}$
else
$x^{(i+1)} = x^{(i)}$

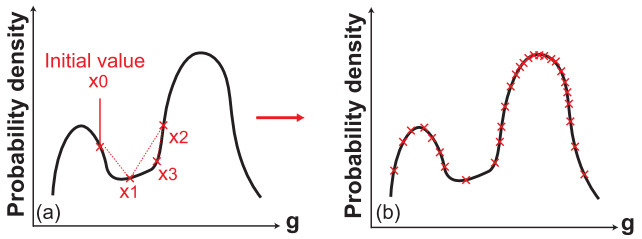


Fig. 2. (a) Posterior distribution, initial value, and new samples drawn from MCMC sampling; (b) final accepted models to represent the distribution.

II. Preliminaries

A. Markov Chain Monte Carlo Sampling Algorithm

MCMC method is one of the fundamental building blocks of probabilistic inference to estimate the posterior probabilities and generate stochastic samples from designated probability distributions [4], especially those whose probability density function (PDF) is hard to analytically express. It is an essential mathematical tool for solving integration and optimization problems in large dimensional spaces [1]. The core idea of MCMC is defining a Markov chain where the integral of state transfer probabilities should form a stationary distribution. In other words, if the chain runs for a long time, the distribution of states will be proportional to the desired distribution.

Metropolis-Hastings (MH) MCMC sampling (Algorithm 1) is the most common approach in probabilistic computing [14]. MH-MCMC is mostly for drawing samples directly from a prior distribution $p(x)$ toward the posterior distribution. Fig. 2 illustrates an MCMC sampling process. Here, x_0 is the initial value as the first sample of the Markov Chain, randomly chosen in the set X of all possible sampling values. Then we randomly draw new samples through the transfer distribution $q(x)$ as new candidates for the following sample. This Markov chain extends smoothly step by step in this way. Finally, we can obtain a group of samples that can represent the stable distribution $p(x)$. In this way, by defining the Markov state transfer probabilities, the MCMC method generates samples of the desired PDF $p(x)$.

An MH step of invariant distribution $p(x)$ and proposal distribution $q(x^*|x)$ involves sampling a candidate value x^* (i.e. generating random number) given the current value x according to $q(x^*|x)$. Then through the calculation of acceptance probability, the Markov chain decides whether to move to x^* or remain at x. The most important part of this process is the generation of $x^{(i+1)}$. And the value of x^*, which may become the value of $x^{(i+1)}$ after the calculation of the accept

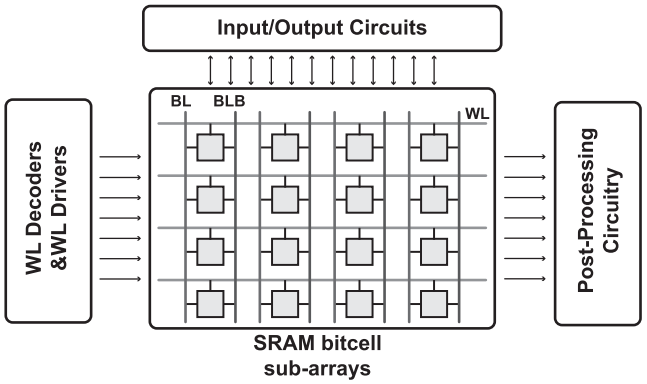


Fig. 3. General circuit architecture of SRAM-based CIM.

ratio, is a random number generated from the previous value $x^{(i)}$ and the transfer function $q(x)$.

An important concept is the "burn-in" [15]. It refers to the process that allows the Markov chain some iterations to reach its equilibrium distribution. The number of iterative cycles (each cycle generates one candidate sample) is called "burn-in time" (or "burn-in period"). Only the samples from the equilibrium distribution of the design Markov chain are meaningful, i.e. we can only use the samples generated after the burn-in time. Ideal calculation of MCMC often uses an empirical burn-in time of 500 or 1000 cycles.

During MCMC calculation, considering the frequent data transfer during iterative random number generation (RNG) and accept-reject check, it is necessary to explore the in-memory implementation of MCMC to significantly improve computing efficiency.

B. SRAM-Based Compute-In-Memory Macro

CIM macro is a novel integrated circuit scheme that directly performs in-situ computational operations in memory. As one of the promising solutions to overcome von Neumann bottleneck [16], [17], [18], [19], [20], this technology has been developed to perform energy-efficient computations in many AI applications [21], [22], [23], [24], [25], [26], [27], [28]. The key idea of CIM is to fuse memory and process units. SRAM is widely used in developing CIM macros for its fast read/write speed and good compatibility with standard CMOS fabrication processes, such as in 28 nm, 7 nm, and 5 nm technology nodes [29]. Fig. 3 shows the basic structure of SRAM-based CIM macro [18]. It consists of 6T-SRAM bitcell sub-arrays and peripheral circuits, including addressing modules, read/write circuits (i.e. sense amplifiers and write drivers), and computing circuits. With the assistance of the peripheral circuits, the bitcell sub-array can achieve the functions of storage and computation in independent working modes. By offloading computation from centralized processing units, CIM dramatically eliminates the frequent data transportation between memory and processing units and thus forms an energy-efficient computing system [30]. Even though CIM technology has been considered to generate random samples [13], it has been rarely investigated in implementing full MCMC accelerator hardware with CIM technology.

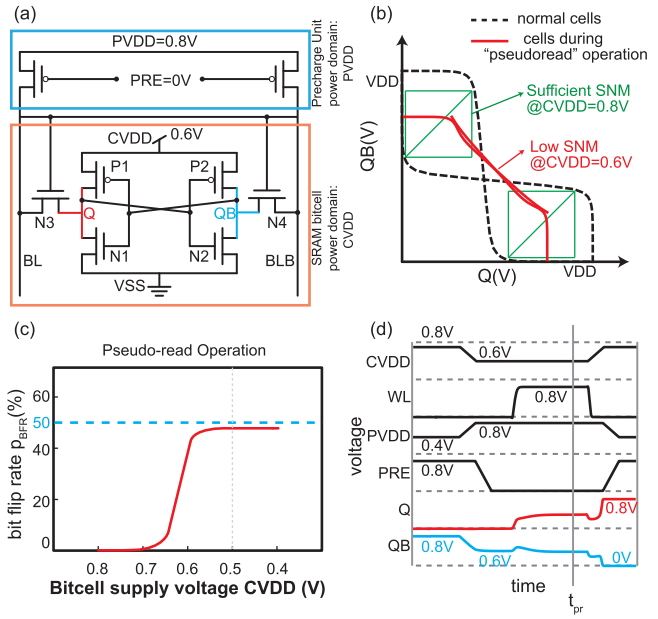


Fig. 4. (a) "Pseudo-read" operation for an SRAM bitcell; (b) VTC of normal SRAM bitcells and the bitcells during "pseudo-read" operation; (c) bit flip rate of SRAM bitcells; (d) working condition and timing diagram for pseudo-read operation.

III. Operating Principles

We propose to exploit the inherent stochastic characteristics of SRAM to develop novel peripheral circuits to realize MCMC hardware acceleration. Before diving into the detailed circuit design, this section provides the basic principles of the proposed CIM-based MCMC design. This work leverages the inherent stochasticity of the SRAM bitcell as the source of randomness. Subsequently, based on this mechanism, the computing process of the CIM-based MCMC design is described in this section.

A. Inherent Stochastic Characteristics of SRAM Bitcell

Data stored in SRAM bitcells can be disturbed by thermal noise. This work would like to leverage this as the source of controllable randomness. According to the SRAM theory in [29], each SRAM bitcell (illustrated in Fig. 4(a)) has its own data retention voltage (DRV), which refers to the minimum supply voltage for an SRAM bitcell to preserve the datum bit resilient to the noise perturbation [31]. One common indicator for the stability of memory bitcells is the voltage transfer curve (VTC), usually called "butterfly diagram", as shown in Fig. 4(b). To quantify VTCs, we simulate with a commercial 28 nm technology PDK and foundry 6T SRAM bitcells. The standard bitcell supply voltage CVDD is 0.8 V. As Fig. 4(b) shows, the edge length of the largest squares inscribed in the "eyes" of the butterfly curve are defined as static-noise margin (SNM) [32]. SNM quantifies the voltage noise necessary to flip the stored datum in a bitcell. When the supply voltage CVDD decreases, the two sides of VTC get close to each other, resulting in a sharp SNM decrease, implying that the bitcell resilience to thermal noise turns weaker. Fig. 4(c) shows the bit flip rate (BFR) [33], [34] throughout this process depending on

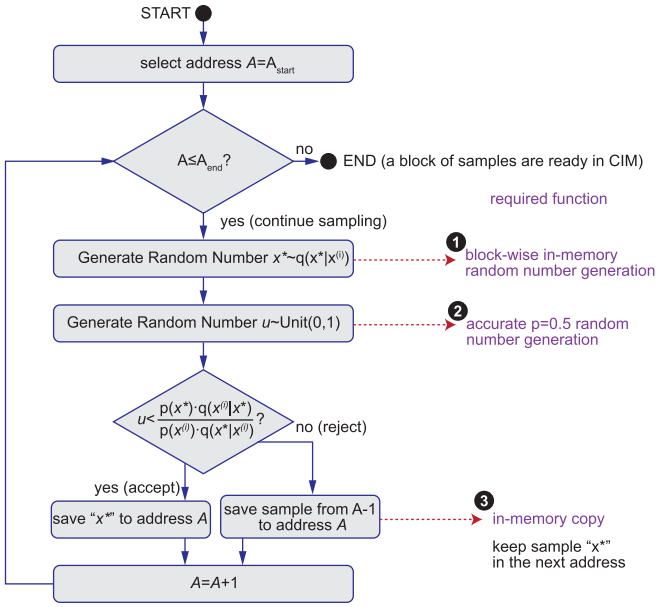


Fig. 5. Block diagram for CIM-based MCMC method.

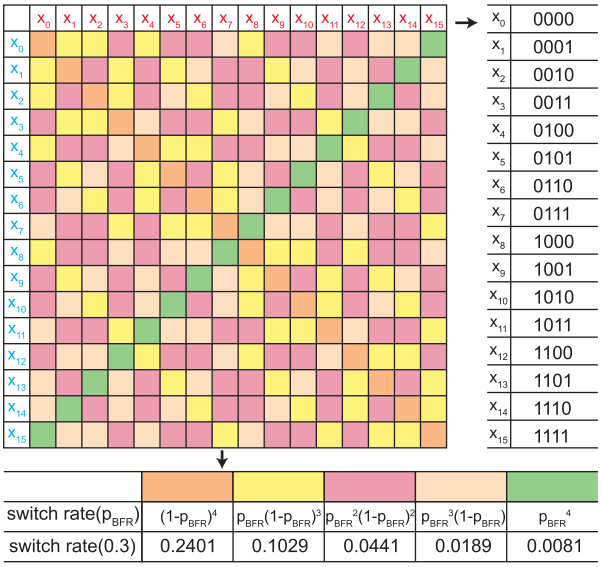


Fig. 6. Probabilistic transfer matrix of 4-bit data.

the bitcell supply voltage CVDD. SNM shrinks as the bitcell datum gains increased chances to flip randomly inflicted by thermal noise.

However, DRV can be very small owing to the back-to-back connection of the two inverters in the SRAM bitcell. When CVDD is around DRV, small changes of CVDD may cause rapid BFR fluctuation due to the circuit nonlinearity. In order to gain better control of the SRAM bit flipping, we propose to define a novel operation, called "pseudo-read". It is an operation similar to SRAM read but destroys the original datum bit stored in the SRAM bitcell and generates a random "0" or "1". Fig. 4(d) shows the working condition and timing diagram for the bitcell pseudo-read. During pseudo-read, we apply 0.8 V to BL and BLB through the bitline conditioning transistors, with a lowered CVDD as the bitcell supply. Then WL gets asserted and later deasserted to trigger the random bit flip phenomenon. e.g. at the time of t_{pr} in Fig. 4(d), the operating configurations are shown in Fig. 4(a). During the pseudo-read, BL and BLB simultaneously keep high to accelerate the SNM shrinking. We observe that BFR is around 45% when CVDD lowers to 0.5 V with this proposed pseudo-read operation.[1] This is adequate as the source of randomness in the proposed MCMC sampling circuits.

B. Computing Steps of CIM-Based MCMC Design

Fig. 5 shows the flow chart of generating stochastic samples with the proposed CIM-based MCMC design. The output of the MCMC is a set of samples $\{x_0, x_1, \cdots, x_m\}$ with required PDF $p(x)$, which are placed in the memory block from the address A_{start} till A_{end}. It consists of three major steps: for each memory address, (a) generate samples x with the transfer PDF $q(x)$; (b) generate one sample u from an accurate [0,1] uniform distribution; (c) check the accept/reject condition: if accepted, the sample should be copied to a new address for results storage.

Here we elaborate on this process with an example of 4-bit samples. Each sample (number) is a value between 0 (0000) and 15 (1111). In this scale, the initial value $x^{(0)}$ is written into 4 SRAM bitcells, each bitcell storing 1 bit. The sampled value x^* follows the random distribution of $x^* \subset q(x^*|x^{(i)})$, and this process is achievable with pseudo-read operation explained in Sec. III-A.

For example, the stored value of 4 SRAM cells is "0" (0000). After the pseudo-read, according to the curve of BFR (Fig. 4(c)), these bitcells have a 45% chance of generating "1". We hereby obtain a sampled 4-bit random number x^*. Then, the next step is to calculate the accept ratio α. Fig. 6 shows the entire transfer matrix of a 4-bit word. Given a certain CVDD for pseudo-read operation, we can measure and plot the transfer matrix $\mathbf{q}$, where we can look up for the values $q(x^{(i)}|x^*)$ and $q(x^*|x^{(i)})$. It is evident to find that this matrix is a diagonal matrix, with $q(i, j) \equiv q(j, i), for\ i, j \subset [0, 15]$. This transfer matrix $\mathbf{q}$ is bound to the condition of pseudo-read, and it is thereby a constant when calculating the accept ratio $\alpha = \frac{p(x^*)q(x^{(i)}|x^*)}{p(x^{(i)})q(x^*|x^{(i)})}$. Subsequently, α is simplified to $\frac{p(x^*)}{p(x^{(i)})}$.

Given the required distribution $p(x)$, this value ("accept ratio") can be easily calculated through peripheral digital logic circuits. With a [0,1] uniformly distributed random number u generated, the comparison between u and the calculated accept ratio decides whether the sample x^* is qualified as the MCMC output sample. According to Step 3 in Algorithm 1, the generated x^*, if accepted, will be copied to the next 4 bitcells for subsequent sampling operations. Otherwise, bitcells that store the unaccepted value will be overwritten the previous random value, and then this value will be copied to the next bitcells. In this way, CIM-based MCMC design fulfils the MCMC computing.

[1]In reality, the value of the lowered CVDD is chosen according to the measured bit flip rate.

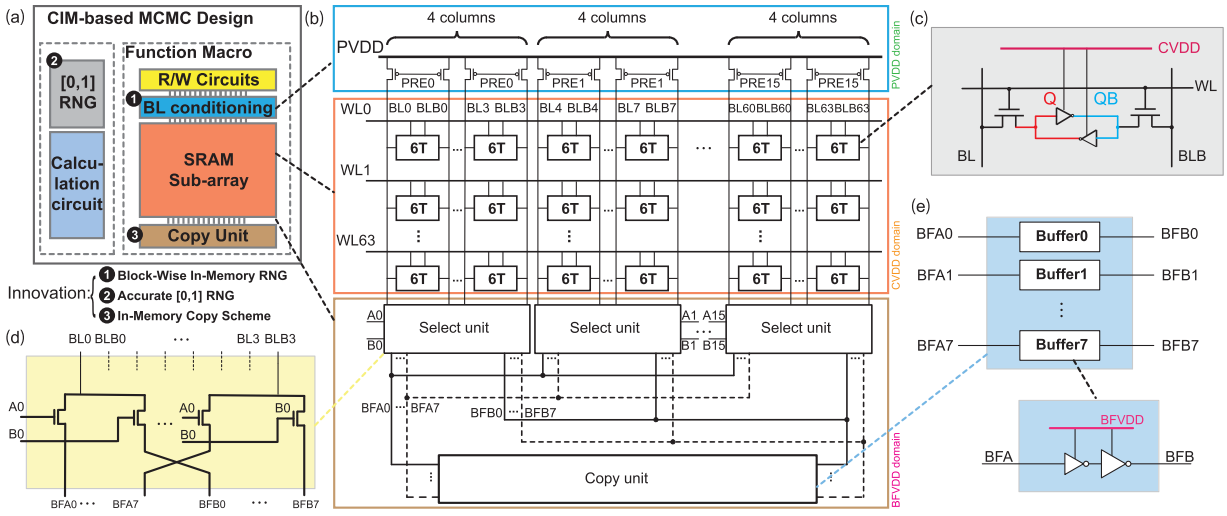


Fig. 7. (a) Architecture of CIM-based MCMC design; (b) sub-array and peripheral circuits; (c) standard 6T-SRAM bitcells; (d) select unit; (e) copy unit.

IV. CIM-Based MCMC Design Hardware Design

As described in Sec. III, CIM-based MCMC computation requires three major in-memory operators: *in situ* in-memory RNG (x^* generator), accurate [0,1] RNG (u generator) and in-memory copy. This section elaborates on the circuit scheme to realize these three operations and form a complete CIM-based MCMC design. CIM-based MCMC design performs on-chip MCMC sampling and storing the generated samples. The entire MCMC processing happens locally inside the CIM-based MCMC design macro circuits, which eliminates the need for frequent data transfer between the processing units and memories in the conventional software realization on CPU/GPU platforms and thus significantly reduces the latency and energy consumption. Fig. 7(a) shows the overall architecture of the CIM-based MCMC design. It consists of an SRAM sub-array,[2] read/write (R/W) circuits, BL conditioning circuits(❶), accurate [0,1] RNG module (❷), and in-memory copy units (❸). Note that the former 4 modules form a function macro. The accurate [0,1] RNG module (❷) is a standalone module independent of the function macro.

This function macro can work in three modes:

- *Memory mode* works the same as traditional SRAM does. The functions can write (sense) data into (from) bitcells. In this mode, the SRAM sub-array, BL conditioning circuits, and R/W circuits work; all the other circuits are turned off.
- *Block-wise RNG mode* generates random numbers into blocks. In this mode, only SRAM sub-array and BL conditioning circuits are turned on.
- *CIM copy mode* realizes copying data from one part to another part. In this mode, only the SRAM sub-array and the in-memory copy unit are turned on.

Next, we will detail how the proposed CIM-based design performs MCMC computing.

[2]The SRAM sub-array is defined to consist of SRAM bitcells, row/column decoders, and multiplexers. The latter two are not drawn here because of their conventional circuit structure.

A. Block-Wise In-Memory Random Number Generation

Practical applications call for vast amounts of random samples before the accept/reject check. According to Fig. 5, a large number of random numbers should be generated and immediately buffered in the memory for further MCMC processing steps. In order to achieve this, we design the 6T SRAM bitcell sub-array and block-wise in-memory RNG assisting circuit, as illustrated in Fig. 7(b). The entire function macro enters block-wise RNG mode. As one realistic setup, every 4 columns of bitcells share one BL conditioning circuit (the precharge unit). The power supply of the BL conditioning circuits is connected to PVDD and the bitcell sub-array connects to the supply voltage of CVDD. In the block-wise RNG mode, pseudo-read[3] is conducted for a block of the sub-array, as shown in Fig. 8(a). The precharge unit of the selected cells is activated and the supply voltage connected to it is raised to 0.8 V with PVDD = 0.8 V, maintaining high voltage on both BL/BLB at the same time. CVDD is reduced to 0.5 V. Then, the WL of the selected cells is activated. As a result, the SNM of these bitcells are reduced purposely (as shown in Fig. 8(b) and (c)), and their values are destroyed. For the bitcells in the same column (Fig. 8(d) and (e)), since the WLs are not activated, their SNM remains high and the value stored is not destroyed [34]. For bitcells in the selected row (Fig. 8(f) and (g)), despite their corresponding WL activated, precharge units connected to them remain turned off. Despite its slight reduction, the SNM of unselected bitcells is still large enough for those unselected bitcells to maintain the stored data. This keeps them from being modified by pseudo-reads. The setting of a separate precharge unit makes it possible for the macro to perform "Random number generate" on specifically chosen cells, guaranteeing that the generation of new samples will not affect previous data that are stored in unselected blocks in the sub-array.

1) Discussion on the Impact of Process Variation: Due to manufacturing process variations, inverters coupled in SRAM bitcells actually do not have 100% symmetrical characteristics.

[3]"Pseudo-read" operation is defined in Sec. III-A.

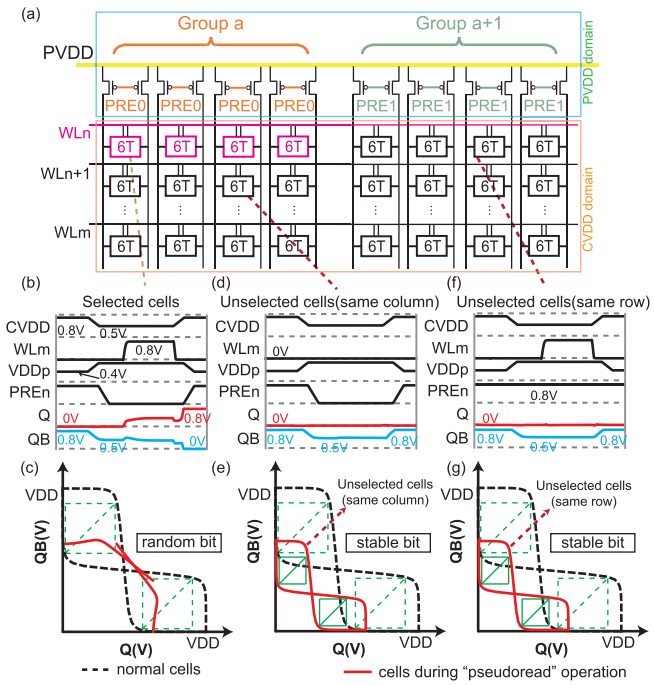


Fig. 8. (a) Pseudo-read operation in a bitcell sub-array; (b) working condition and timing diagram of the selected bitcells; (c) VTC of the selected bitcells and unselected bitcells; (d) working condition and timing diagram of the unselected bitcells in the same column of the selected bitcells; (e) VTC of the unselected bitcells in the same column of the selected bitcells; (f) working condition and timing diagram of the unselected bitcells in the same row of the selected bitcells. (g) VTC of the unselected bitcells in the same row as the selected bitcells.

However, in practical calculations, this suboptimal characteristic does not impact the specific operation of the circuit. Randomness plays a dominant role in the algorithm of Markov Chain Monte Carlo sampling. The subtle errors between SRAM bitcells can be viewed as introducing a small amount of environmental noise into the process of generating a Markov chain, typically having little impact on the results of MCMC algorithms.

B. Accurate [0,1] Random Number Generation

After the candidate samples for the desired Markov chain are obtained from the block-wise RNG, whether they can be accepted as the output samples depends on the accept/reject check step. That is, compare a random number u and the expression $\alpha(x^{(i)}, x^*)$ in Algorithm 1. This requires the random number u to be uniformly distributed between [0,1].

The challenge of realizing this accurate [0,1] RNG is twofold: (a) software [35] or digital-logic-based pseudo-random number generation algorithms potentially cause large area and energy overhead [36]; (b) since the SRAM bitcell BFR p_{BFR} under pseudo-read operation is lower than 50% as shown in Fig. 4(c), direct use of pseudo-read cannot guarantee that the generated random number u is uniformly distributed,[4] which potentially bring adverse effects on the quality of MCMC sampling. Therefore, we propose a novel low-cost multi-stage XOR gate circuit scheme (MSXOR) to construct the accurate [0,1] RNG.

Fig. 9(a) depicts the circuit schematic diagram of the proposed accurate [0,1] RNG. Accurate [0,1] RNG includes a small-scale 6T SRAM bitcell sub-array and a group of multi-stage XOR gates. This small-scale SRAM bitcell sub-array has a similar structure to the sub-array used block-wise RNG in Sec. IV-A; yet the former has no R/W circuits. Besides, in an accurate [0,1] RNG module, the voltage VDDP is divided into two voltages VDDPL and VDDPR to respectively control BL and BLB.

This scheme requires two steps to generate a multi-bit accurate [0,1] random number:

Step 1: Flush the employed SRAM bitcells to zeros ("reset"). This step can be performed through the voltage difference of VDDPL and VDDPR, as illustrated in Fig. 9(b). VDDPL and VDDPR are both controlled by the transistors MP0∼MP127 with the same control signal PRE. In the reset operation, the supply voltage of the SRAM bitcells is firstly reduced to a relatively low value (0.5V), and then the supply pair (VDDPL, VDDPR) is set to (0 V, 0.8 V). Then the MP0∼MP63 and WL are turned off. Since VDDPR is higher than the bitcell supply voltage, the stored value of these bitcells is modified to "0".

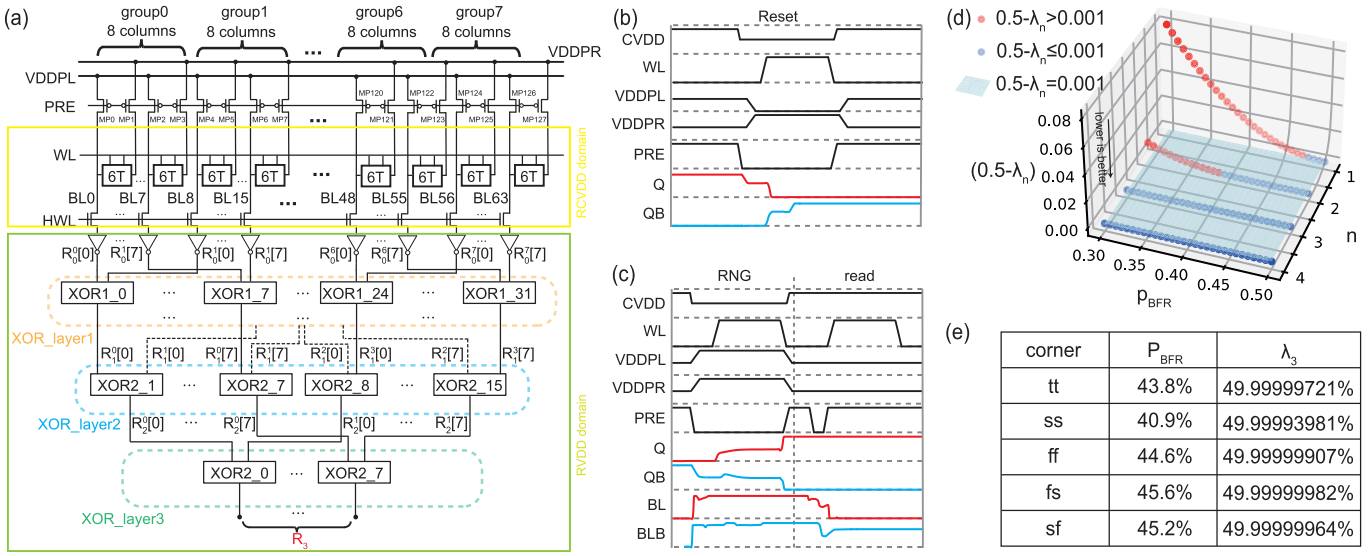
Step 2: Perform the pseudo-read RNG operation on these bitcells, which is similar to the step described in Sec. IV-A. Considering the separate VDDPL and VDDPR controlling BL/BLB, both VDDPL and VDDPR should be raised to 0.8 V in the pseudo-read steps. The waveform of the control signals is depicted in Fig. 9(c).

Next, we will show why this scheme with the two-step operation yields samples of uniform distribution. First, we define the probability of a single bit to be "1" as λ. The multiple stages of XOR gates help optimize λ of the generated samples to 0.5 and equivalently the aggregated multi-bit results achieve uniform distribution. In the circuit scheme, 64 bitcells are divided into 8 groups, i.e. each group representing an 8-bit "raw" random number $R_0^l[7:0]$ ($l \in [0,7]$). Note that the probability of each bit in $R_0^l[7:0]$ as "1" is $\lambda_0 = p_{BFR}$ because it is right out of reset and pseudo-read operation. The subscripts of R_0^l and λ_0 mean it is the first stage.

Then it feeds $R_0^0, R_0^1, \cdots, R_0^7$ into a stage of 64 XOR gates. Each XOR gate converts two input bits into one output bit. Therefore, at this stage, the XOR gates output R_1^0, R_1^1, R_1^2, R_1^3. As illustrated in Fig. 9(d), both the two inputs at this stage have a probability of λ_0 being "1" and a probability of $1-\lambda_0$ being "0". The output bit of the XOR gate hereby has a probability of $\lambda_1 = 2\lambda_0(1-\lambda_0)$ to be "1" (and a probability of $(1-\lambda_1)$ being "0").

Then R_1^0, R_1^1, R_1^2, R_1^3 successively pass stage 2 (32-bit XOR gates) and stage 3 (16-bit XOR gates). Finally, this module yields 1 group of samples $R_3[7:0]$ as the final output of accurate [0,1] RNG in 8-bit. Take $p_{BFR} = 0.4$ as an example, $\lambda_3 = 0.49999872$. The detailed proof of $\lim_{n\to\infty} \lambda_n = 0.5$ can be found in Appendix B. Fig. 9(d) shows the quantitative analysis result of the distance from $\lambda = 0.5$, i.e. $(0.5 - \lambda_n)$, given the value of p_{BFR} and the number of XOR gate stages n. It reveals that when $p_{BFR} \geq 0.4$ (corresponding to the case of $CVDD$

[4]For a multi-bit integer u, uniform distribution can be interpreted as that each binary bit of u has a strict 50% probability of being "1" or "0".



corner	P_{BFR}	λ_3
tt	43.8%	49.99999721%
ss	40.9%	49.99993981%
ff	44.6%	49.99999907%
fs	45.6%	49.99999982%
sf	45.2%	49.99999964%

Fig. 9. (a) Proposed accurate [0,1] RNG architecture; (b) working condition and timing diagram of the reset operation; (c) working condition and timing diagram of RNG and read operation; (d) λ of different bit flip rate and number of XOR_stages; (e) λ_3 of SRAM bitcells under different corner simulations.

is disturbed from 0.5V to 0.6V), 3-stage XOR-gates ($n = 3$) are adequate.

XOR gates are often used to detect phase differences in the field of communication [37], whereas this work uses the multi-stages of XOR gates to tune the probability from non-uniform distributions into a uniform one. The signals that XOR gates process in this work are static, i.e. the information is not encoded in the timing of the signals. This method serves as an effective solution for the accurate [0,1] RNG as required by the MCMC acceleration.

1) Discussion on the Impact of Process Variation: as the [0,1] RNG circuit is composed of a limited number of SRAM bitcells, process variation of these bitcells is an adverse factor that potentially affects the accuracy of uniform distribution. Moreover, actual inverters coupled in SRAM bitcells do not have 100% symmetrical characteristics. The asymmetry factors may also have an impact on the performance of this [0,1] RNG circuit. To address this issue, this work carries out the corner simulation for the accurate [0,1] RNG circuit. The simulation uses CVDD = 0.5V in pseudo-read as the controlled variable. Fig. 9(e) summarizes the corresponding results. It reveals that p_{BFR} fluctuates in different design corners as influenced by the process variation. This fluctuation arises because the operating period may not be appropriate for the SRAM bitcells to reach an intermediate voltage level during pseudo-read. Our simulation shows that extending the latency of pseudo-read operation can ameliorate this problem. Fig. 9(e) shows that the proposed 3-stage XOR gates yield $\lambda_3 \geq 0.4999993981$, which is adequately accurate for the uniform distribution RNG and thus passes the corner test.

After a candidate sample x (from the block-wise in-memory RNG) and a uniformly distributed random number u (from accurate [0,1] RNG) are prepared, the accept ratio calculation is then performed with digital arithmetic units. First, convert the random number u into the range of [0,1]. Since R_3 is an uniformly distributed 8-bit number between $0(0 \times 00)$ and $255(\texttt{0xff})$. The desired random number can be calculated

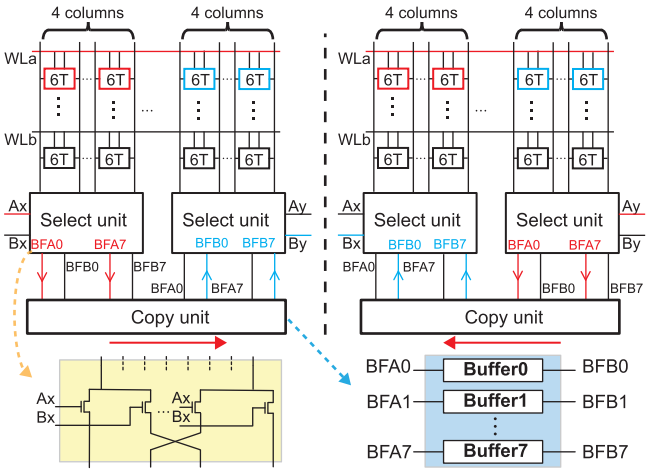


Fig. 10. In-memory copy scheme.

through $u/256$. Then calculate the expression $\frac{p(x^*)q(x^{(i)}|x^*)}{p(x^{(i)})q(x^*|x^{(i)})}$ and compare the value with u. To Reduce the complexity of the program, we can change the algorithm from comparing the size of fractions to comparing representation $u * p(x^*)$ and $p(x^{(i)})$. If $p(x^{(i)}) > u * p(x^*)$, the new sample should accepted. In this case, an in-memory copy operation should be carried out to store the accepted sample.

C. In-Memory Copy Scheme

After obtaining an accepted sample, it should be written to another nearby address for further use. A normal "read-and-write" operation through SRAM R/W circuits to accomplish this is straightforward but time-consuming. Because the value to be written back to the array is exactly stored in the previous group of bitcells, a method of copying data between neighboring bitcells could be a better solution for reducing read-write latency and power consumption.

To achieve this, we propose an in-memory copy scheme, as shown in Fig. 10. Every 4 columns of bitcells in the

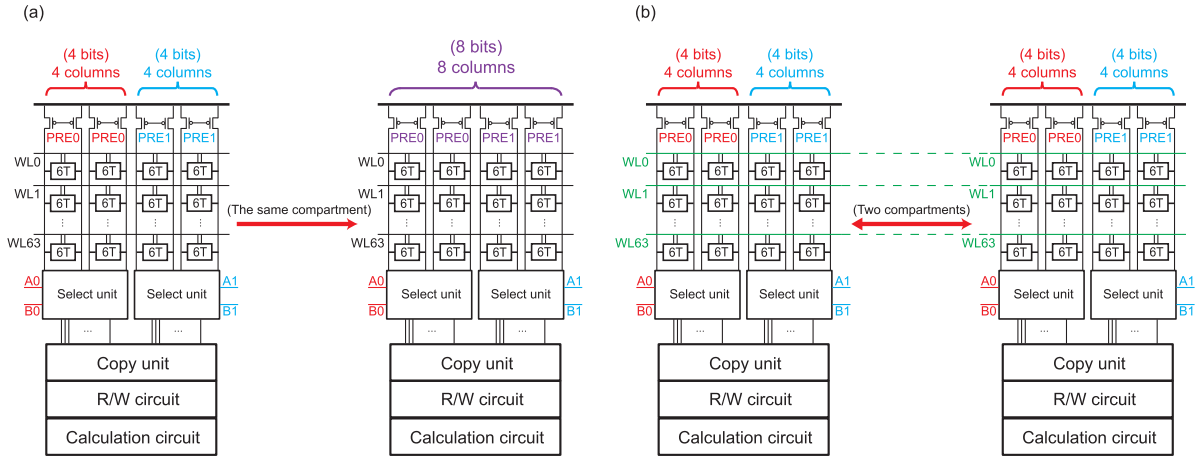


Fig. 11. (a) Expandable precision scheme; (b) scalable parallelism scheme.

sub-array form a group with two control signals A and B. Note the 4-column configuration follows Fig. 7(b). Every compartment of the macro shares 8 buses, which correspond to BLs and BLBs from every group of bitcells. Every bus has one buffer,[5] whose supply voltage remains high (0.8 V). Since the buffer is unidirectional, every BL (BLB) is connected to both sides of the corresponding buffer with signal A controlling the switch on the input port and B controlling the switch on the output port. To copy the value from bitcells from columns $[a_0, b_0, c_0, d_0]$ to columns $[a_1, b_1, c_1, d_1]$ in the same row, the supply voltage of the sub-array is reduced to 0.5 V. Then the WL of this row is activated and the control signal $A0$ corresponding to $[a_0, b_0, c_0, d_0]$ on the buffer input port and $B1$ corresponding to $[a_1, b_1, c_1, d_1]$ on the buffers' output port are raised. In this way, the values of the bitcells in $[a_0, b_0, c_0, d_0]$ are transferred to the voltage on BLs and BLBs and enlarged to 0.8 V through the associated buffers. They hereby are written into the bitcells of $[a_1, b_1, c_1, d_1]$ with a relatively low supply voltage. After a short writing latency ($<$1 ns in our experimental configuration), the WL turns off, and the supply voltage of the whole array restores 0.8 V. In this way, an efficient in-memory copy operation is realized.

V. Enhancement for CIM-Based MCMC Design

The proposed CIM-based MCMC design is expandable in the precision of generated samples and scalable for high parallelism.

A. Expandable Precision

Earlier, when we describe the working principle of the proposed macro, we take the example of a word composed of 4 bits. Actually, the proposed CIM-based MCMC design supports higher precision as required in some applications [38]. In this way, we take a precision-expandable scheme to solve this problem. As shown in Fig. 11(a), the array described in this paper is divided into several compartments, with each compartment composed of 4096 SRAM cells (64 rows & 64 columns). Since in the most basic case these bitcells are divided into 16 groups and each group consists of 4 columns, we can combine 2 groups of bitcells into a new group by keeping their control signal at the same pace with each other. When performing the sampling, these control signals are turned on simultaneously, enabling outputting an 8-bit sample at a time. Under R/W operations and in-memory copy operations, we can apply the principle of separate transmission, operating on the first 4 columns and then the next 4 columns. Through this method, we can conduct in-memory copy for the samples of up to 64 bits.

[5]Here the buffer refers to a circuit buffer, i.e. even number of inverters to increase the drivability.

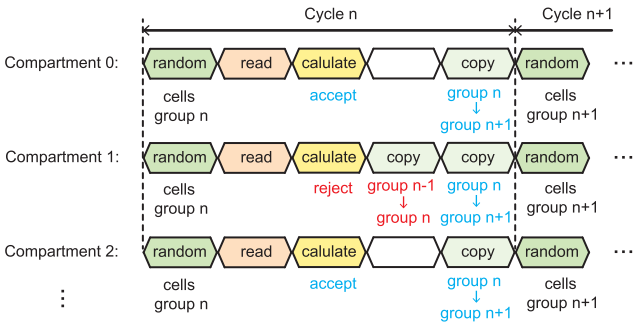


Fig. 12. Operating timing diagram of this probabilistic macro.

B. Scalable Parallelism

For computing, high parallelism is what researchers always pursue. The proposed one macro can only perform MCMC sampling once at a time, failing to deliver adequate parallelism. Therefore, we propose to improve the parallelism scheme by dividing the whole sub-array into different compartments. Fig. 11(b) depicts this scheme. Each compartment is composed of 4096 SRAM bitcells (64 rows, 64 columns), copy circuit for the in-memory copy scheme, R/W circuit, and calculation circuits. When we perform sampling, every compartment is thrown into operation. The working timing diagram of a compartment-based macro is shown in Fig. 12. For every step ("random" means block-wise RNG, "read" means reading out the generated sample and performing the accurate [0,1] RNG simultaneously, "calculate" means the calculation of

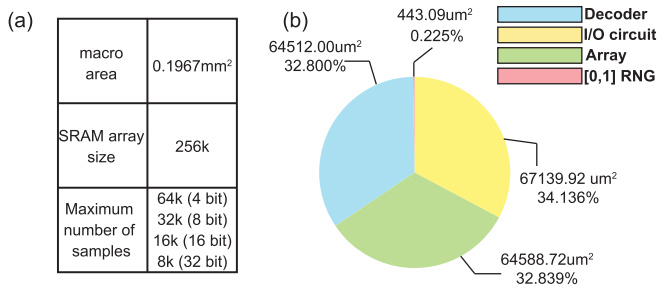


Fig. 13. (a) Design parameter of the proposed macro; (b) the total area breakdown of this macro.

accept/reject check, and "copy" means), WLs of all compartments and corresponding control signals are turned on simultaneously so as to output one sample per compartment instead of per entire macro. The acceptance probability calculation operation is performed at the same time for every compartment. Of course, some samples will be accepted and others will be refused. Therefore, two independent groups of in-memory copy circuits can solve this problem. In one group of in-memory copy circuits, the compartments that obtain accepted samples do not need to modify themselves, as the remaining should copy the previous sample to the present address. This operation can also be performed simultaneously, with WLs of the accepted compartments turned off and the remaining WLs turned on. In the other group of in-memory copy circuits, all compartments are involved, with the value of bitcells of the present address copied to bitcells of the next address. In this way, a group of newly sampled data is obtained and the Markov Chain keeps evolving [19].

VI. Evaluation and Discussion

A. Configuration for Verification & Circuit Implementaiton

The proposed CIM-based MCMC scheme is designed in the latest commercial 28 nm CMOS process development kit. The 6T SRAM bitcell is the foundry-developed bitcell. The capacity of this macro is 256 kb and the core area is 0.1967 mm^2. Fig. 13(a) lists the design parameter of this macro.

The area is estimated based on the layout of prior practical SRAM-based CIM design [18]. Fig. 13(b) presents the pie diagram for the entire area breakdown. Generally, R/W circuits dominate the area of this macro (34.136%). SRAM sub-array with select and copy units is the second largest part of this macro (32.839%). It is slightly larger than the WL decoders (32.800%). Since one random number generated from accurate [0,1] RNG can be shared by the overall 64 compartments. The proposed accurate [0,1] RNG is very compact, only occupying 0.225% of the entire area.

B. Function Verification

In this section, the function of the proposed macro is verified. Fig. 14 shows the simulated timing diagram of SRAM bitcells as well as the corresponding output of R/W circuits.

Fig. 14(a1-a4) correspond to SRAM bitcells belonging to the first 4 columns, i.e. columns (0,1,2,3). Fig. 14(b1-b4) correspond to the second 4 columns, i.e.columns (4,5,6,7). Fig. 14(c1-c4) corresponds to the output of the R/W circuit. "Q" refers to the positive terminal inside the 6T SRAM bitcell in Fig. 7(b).

- From 0 ns to 1 ns (x-axis), the proposed macro goes through the "write" process, writing an exemplary value "0101"into the 4 SRAM cells belonging to columns (0,1,2,3).
- From 1 ns to 2 ns, block-wise RNG is performed (marked with"random" in Fig. 14). The originally stored data changes from "0101" to "1001". This new data "1011" serve as the first sample of the Markov chain.
- From 2 ns to 4 ns, the macro executes "in-memory copy". The data "1001" stored in columns (0,1,2,3) are copied to the bitcells of columns (4,5,6,7).
- From 4 ns to 5 ns, the macro executes block-wise RNG on the bitcells of columns (4,5,6,7), with their stored data switching from "1001" to "0100".
- From 5 ns to 6 ns, the new data "0100" is read through the output circuit for the accept/reject check.

In this way, the proposed macro gets a group of samples stored in the SRAM array following the desired distribution with the MCMC method.

C. Thermal Impact

The operation of the proposed macro strongly depends on SRAM bitcell stochastic behavior [39]. The stochasticity is strongly related to the ambient temperature. Different temperatures may affect the working efficiency of the random generation. Keeping the supply voltage 0.5 V, we measure bit flip rate p_{BFR} at different temperatures. As shown in Fig. 15, p_{BFR} slightly increases as the temperature rises. In the normal commercial-grade operating temperature range$0°C - 70°C$, p_{BFR} remains around 45%. This implies the influence of the ambient temperature. For industrial grade $(-40°C - 85°C)$, this macro can work in most temperature ranges $(-20 - 85°C)$ with a stable bit flip rate. When working at a lower temperature $(-40 - -20°C)$, p_{BFR} decreases as thermal noise reduces. According to the prior study in [1], the decrease in bit flip rate will only result in an extension of burn-in time to generate random samples without affecting the quality of obtained samples. For the accurate [0,1] RNG, according to Fig. 9(d) and (e), a slight decrease of p_{BFR} is tolerable for maintaining the correct RNG function.

D. Energy and Efficiency

Fig. 16(a) shows the energy consumption of the proposed macro under different operations. When performing the block-wise RNG and in-memory copy operations, this macro consumes 79.1 fJ and 47.5 fJ for each 4-bit sample, respectively. Generally, the operations involving R/W circuits consume more energy than those completely inside the SRAM sub-array (343.1 fJ for read operation and 372.6 fJ for write operation). This again confirms the necessity and importance of the in-memory copy scheme. By avoiding excessive reuse of the R/W circuit in frequent data transportation, the proposed

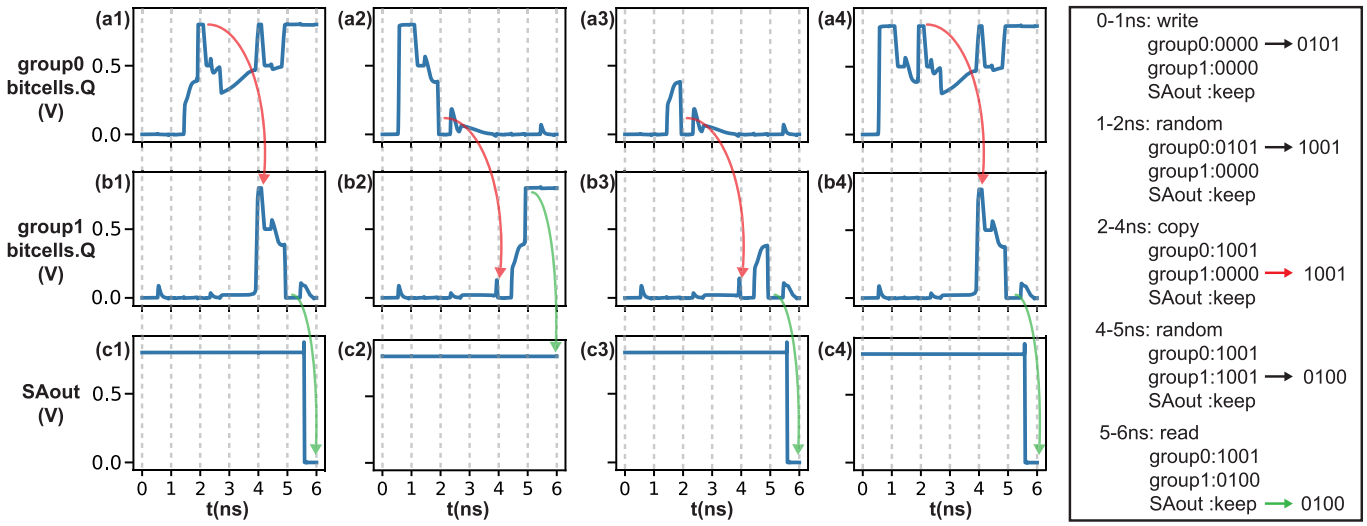


Fig. 14. Function realization of the proposed macro.

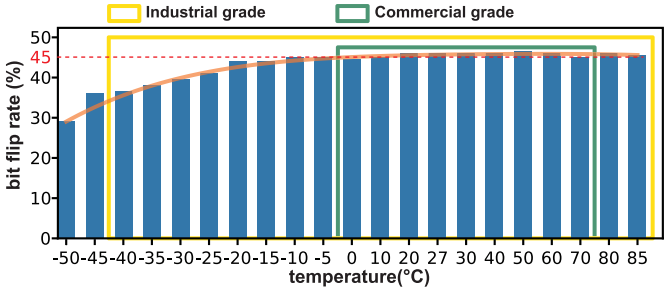


Fig. 15. Bit flip rate under different temperatures.

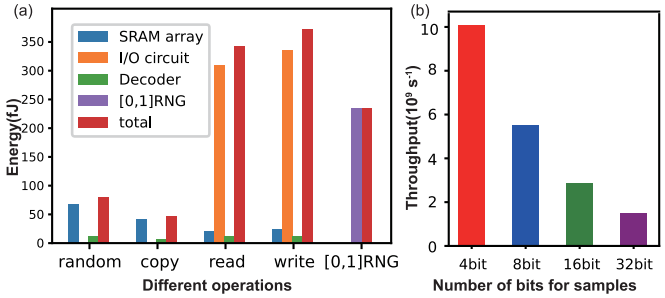


Fig. 16. (a) Energy consumption and breakdown of different operations; (b) throughput of the proposed macro with different precision of samples.

macro significantly reduces energy consumption in accelerating MCMC computation. The energy consumption of [0,1] RNG is 234.6 fJ for each 8-bit sample.

Considering everything, the energy cost of this entire macro is 0.5065 pJ/accepted sample for the sampling operation, whose result is a directly accepted sample. However, suppose the calculation circuit rejects the obtained sample. In that case, the energy cost of this operation will deteriorate as we should conduct an extra in-memory copy operation to rewrite the value of the previous sample to this rejected sample. In this case, the energy cost is 0.5547 pJ/sample. The sampling accept ratio typically remains between 30% and 40% based on different prior distributions. Generally, considering the overall acceptance, the energy efficiency of the proposed macro will range from 0.5331 pJ/sample to 0.5402 pJ/sample.

E. Throughput

Fig. 16(b) illustrates the throughput of the proposed macro with different precision of samples. As the number of bits in the sample increases, the overall throughput continuously decreases but still maintains above 10^9 samples/s. It proves the proposed macro keeps a high sampling speed. It is notable that as the number of bits doubles, the overall throughput of this macro does not decrease precisely by half. It is slower because when we perform sampling operations with different precision, in-memory copy, R/W operations are performed step by step with 4 SRAM bitcells. As the number of bits doubles, the time consumed in these operations will also double. However the in-memory random operation can be performed simultaneously for any number of bits, as the selected SRAM cells are always in the same row, and their WL is opened consistently. By adjusting pre-charge units of different columns, we can realize the generation of a complete sample in one sample operation, eliminating the time consumed to proceed in steps. Thus, generally, it takes less than twice the time to generate a sample of double the number of bits. The overall throughput will accordingly decrease slower than decline by half.

F. Comparison to MCMC Benchmarks on CPU&GPU

In this part, we compare the performance of the proposed macro with CPU and GPU on different sampling tasks. We designate two probability distributions commonly used in artificial intelligence systems for MCMC, as illustrated in Fig. 17(a) and (b). The first one is the Gaussian mixture model (GMM) [40] in Fig. 17(a), a probabilistic model widely used for representing the distribution of data points in a dataset. It assumes that the dataset is generated from a mixture of several Gaussian distributions, each characterized by its mean, covariance, and weight. The mean represents the center of this distribution; the covariance captures its shape and orientation,

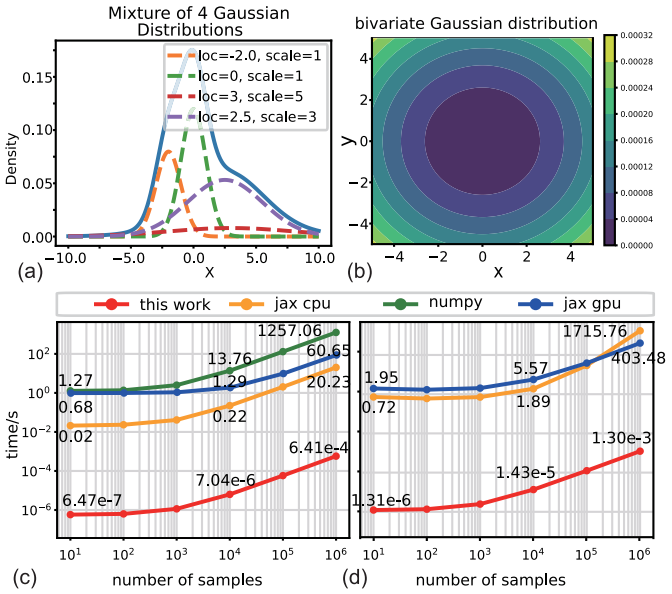


Fig. 17. (a) Mixture of 4 Gaussian distributions; (b) two-dimensional heat map for a bivariate MGD example; (c) experimental result of the GMM case; (d) experimental result of the MGD case.

and the weight indicates the contribution of each component to the overall mixture. Here we use a mixture of 4 Gaussian distributions for GMM. The other one is multivariate Gaussian distribution (MGD) [41], also known as a multivariate normal distribution. It is a probability distribution that generalizes the concept of the Gaussian distribution to multiple dimensions. Fig. 17(b) shows the two-dimensional heat map for a bivariate MGD example. Darker colors represent lower probabilities, and lighter colors represent higher probabilities. This distribution is widely used in tasks such as pattern recognition [42] and anomaly detection [43].

In the experiment, we used CPU, GPU, and simulation results of the proposed macro to test the sampling efficiency on these two distributions. For GMM, we performed sampling on the CPU with the NumPy [44] and its adapted algorithms. Additionally, we employ JAX library [45] with just-in-time compilation and executed along for faster speed on both CPU and GPU. The CPU hardware platform is Intel® Xeon® Gold 6230 CPU @2.10GHz with 220 GB main memory. The GPU hardware platform is NVIDIA® GeForce® RTX 3090, and the samples drawn from our probabilistic macro are fixed at 32 bits.

The experimental result is shown in Fig. 17(c) (GMM case) and Fig. 17(d) (MGD case). The x-axis is the target number of sample generation using the MCMC algorithm and the y-axis is the time cost for completing the corresponding MCMC tasks.

For software algorithms, two points need to be explained. The first is that the sampling speed of JAX of CPU is faster than GPU when performing sampling for GMM. This is because GMM is a relatively simple distribution, and the number of samples is rather low. When performing large-scale, highly parallel computations, GPUs have an advantage over CPUs, but this advantage does not serve totally for sampling GMM. When we perform sampling on MGD, the speed advantage of GPUs over CPUs becomes evident as the number of samples increases. The second is we did not use NumPy algorithms when performing GCD sampling. This is because the NumPy library is generally used for handling relatively simple computations, and it has been proven to be challenging to use NumPy and related algorithms for sampling on MGD.

Fig. 17(c) compares the sampling speed of this macro with different algorithms on CPU and GPU on GMM. The sampling algorithm using the NumPy library on the CPU takes more than 1200 s (20 min) to generate 10^6 samples. Using the JAX library significantly improves sampling efficiency [45], but the sampling time remains more than 10 s on both CPU and GPU. Our proposed macro boosts sampling throughput, with 10^6 samples generated within 10^{-3} s, showing an acceleration of over $3,0000\times$.

Fig. 17(d) shows the sampling speed on a bivariate MGD. The sampling speed shows a noticeable decrease compared to GMM as the distribution grows more complex. As the number of samples increases, the advantage of GPU over CPU in data processing becomes evident. However, the JAX algorithm based on GPU still takes around 400 seconds to generate 10^6 samples. On this complex sampling task, the superiority of our chip is further demonstrated, as the proposed macro only takes around 1.3 ms to draw 10^6 samples.

The power efficiency of the sampling of the proposed macro is also significantly higher than the GPU. Both GMM and MGD benchmarks generate given numbers (ranging from 10^1 to 10^6) of 32-bit samples. In the GMM benchmarking, the average power consumption of GPU is around 125 W, compared to 5.59 mW on our chip. In the MGD benchmarking, the average power consumption of the GPU is around 170 W, compared to 5.52 mW on our chip. Taking sampling speed and power efficiency into account, the overall energy cost of obtaining one sample of our chip is 2.12×10^9 to 9.58×10^9 times lower than that of GPU.

There are two reasons why the performance of our chip is significantly better than that of a GPU. Firstly, our chip is dedicated to MCMC sampling. We leverage the inherent stochastic characteristic of SRAM bitcells to generate random numbers, which is inherently energy-efficient. Additionally, we employ a novel in-memory copy scheme, significantly reducing the energy consumption in generating samples from the Markov chain. Whereas the hardware random-number generator in GPU commonly uses linear feedback shift registers (LFSRs) to generate pseudo-random numbers. The pseudo-random number sequence generated by LFSRs is periodic, and the length of this period increases with the number of D flip-flops in the LFSR. To extend this period and prevent the occurrence of repeated random numbers, GPU often employs a large number of LFSRs for pseudo-random number generation. This will greatly increase the power consumption of GPU for generating random samples in the process of MCMC. Moreover, we adopt the compute-in-memory architecture to perform the sampling steps within the memory. Compared to GPU, this work greatly reduces the energy consumption associated with data transportation between memory and processors.

TABLE I
COMPARISON OF MCMC ACCELERATOR DESIGNS

Work	[10]	[11]	[12]	[13]	This work
Hardware	FPGA	FPGA	ASIC (45 nm)	ASIC (28 nm)	ASIC (28 nm)
Distribution	Gaussian	Gaussian	Gaussian mix-ture models	Arbitrary	Arbitrary
Precision	8-bit	32-bit	8-bit	8-bit	4/8/16 /32/64-bit
Speed	4.7 ns (RLF) 8.5 ns (Wal-lace)	2.5 ns	4 ns	Not given	0.09 ns (4-bit) 0.64 ns (32-bit)
Energy efficiency /sample	2.48 nJ (RLF) 4.76 nJ (Wal-lace) (8-bit)	38.25 ±8.50 nJ (32-bit)	0.36 pJ (8-bit)	3.6 μJ (8-bit)	0.53 pJ (4-bit) 3.58 pJ (32-bit)

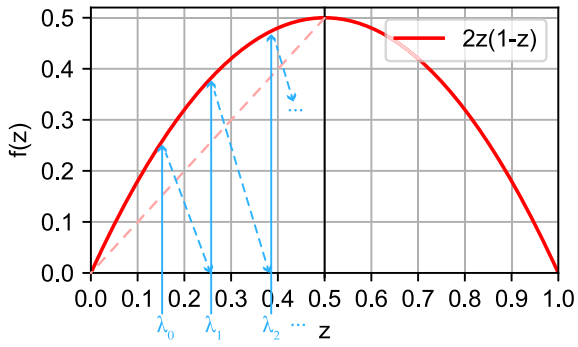


Fig. 18. $f(z) = 2z(1-z)$ and iteration of λ_n.

G. Comparison to Related Works

A comparison with other MCMC accelerating designs is summarized in Table I. Benefiting from the fast in-memory random number generation method and novel in-memory copy scheme, our work achieves sub-nanosecond latency for sampling operations at configurable sample precision. Moreover, with the help of the energy-efficient CIM architecture, the energy efficiency of our work reaches < 0.15 pJ per bit for each sample.

VII. CONCLUSION

This work proposes an SRAM-based compute-in-memory macro for Markov Chain Monte Carlo (MCMC) sampling. Featuring high sampling speed, expandable precision, and high parallelism, this CIM macro can realize the complete MCMC sampling process, including on-chip sampling, random number generating, accept ratio calculation, and in-memory data copy. Implemented in a standard 28-nm CMOS process, this macro can achieve high energy efficiency of 0.53 pJ/sample and high throughput of up to 1066.7M samples/s. Generally, our chip has a power consumption that is 4 orders of magnitude lower than that of a GPU.

APPENDIX A
LIST OF ALL VARIABLES

Variable	**Description**
$p(x)$	the prior distribution
$q(x)$	the transfer distribution
t_{pr}	time to trigger the random bit flip phenomenon
X	set of all possible sampling values
x^*	newly sampled candidate value
x^i	value accepted and stored in the previous address
x^{i+1}	value accepted and stored in the present address
A_{start}	the starting address for the target
A_{end}	the ending address for the target
A	present address
u	random number used in accept ratio calculation
α	the accept ratio
p_{BFR}	bit flip rate of SRAM bitcells
λ	probability of bit as 1 in a generated multi-bit sample
R_n^l	the lth generated sample in the nth stage in the multi-stage XOR gate scheme

APPENDIX B
PROOF OF $\lim_{n\to\infty} \lambda_n = 0.5$

The proposed multi-stage XOR gate scheme $\lambda_{i+1} = 2\lambda_i(1-\lambda_i)$. The function is abstracted as $f(z) = 2z(1-z)$. The curve of $f(z)$ is shown in Fig. 18.

Theorem 1: $\forall 0 < z < 0.5$, $f(z) > z$.

Proof: $\forall 0 < z < 0.5$, $f(z) - z = 2z(0.5-z)$. $\because z > 0$ and $0.5 - z > 0$, $\therefore f(z) > z$. □

Theorem 2: $\forall 0 < \lambda_0 < 0.5$, $\lim_{n\to\infty} \lambda_n = 0.5$.

Proof: Considering Theorem 1, $\forall 0 < \lambda_0 < 0.5$, $\therefore \lambda_0 < \lambda_1 < 0.5$. For next iteration, $\because 0 < \lambda_0 < \lambda_1 < 0.5$, $\therefore 0 < \lambda_1 < \lambda_2 < 0.5$. Thus, it can be easily deduced that,

$$\forall n \in \mathbb{N} \& n \geq 2, \quad 0 < \lambda_{n-1} < \lambda_n < 0.5 \tag{1}$$

And for each iteration, the increment $\Delta(\lambda_n) = \lambda_n - \lambda_{n-1} = \lambda_{n-1} - 2\lambda_{n-1}^2$. Because $0 < \lambda_{n-1} < \lambda_n < 0.5$, therefore

$$0 < \Delta(\lambda_n) \leq 0.125. \tag{2}$$

We can use proof by contradiction here. Suppose $\lim_{n\to\infty} \lambda_n = \Lambda$, where $\Lambda \neq 0.5$. Then, according to Eq. 1, Λ must be < 0.5. When $n \to \infty$, $\Delta(\Lambda)$ has to be $\to 0$. But this is contradicted to (Eq. 2).

Therefore, with $n \to \infty$ (large enough), $\lim_{n\to\infty} \lambda_n = 0.5$. □

Note that the reset operation before pseudo-read in the accurate [0,1] RNG is to guarantee $0 < \lambda_0 = p_{BFR} \leq 0.5$.

REFERENCES

[1] C. Andrieu, N. De Freitas, A. Doucet, and M. I. Jordan, "An introduction to MCMC for machine learning," *Mach. Learn.*, vol. 50, no. 1, pp. 5–43, Jan. 2003.

[2] R. M. Neal, "Markov chain sampling methods for Dirichlet process mixture models," *J. Comput. Graph. Statist.*, vol. 9, no. 2, pp. 249–265, Jun. 2000.

[3] C. Karras, A. Karras, M. Avlonitis, and S. Sioutas, "An overview of MCMC methods: From theory to applications," in *Proc. IFIP Int. Conf. Artif. Intell. Appl. Innov.* Cham, Switzerland: Springer, 2022, pp. 319–332.

[4] Z. Ghahramani, "Probabilistic machine learning and artificial intelligence," *Nature*, vol. 521, no. 7553, pp. 452–459, May 2015.

[5] E. Chung, Y. Efendiev, W. Tat Leung, S.-M. Pun, and Z. Zhang, "Multi-agent reinforcement learning accelerated MCMC on multiscale inversion problem," 2020, *arXiv:2011.08954*.

[6] Y. Chen, S. Huang, T. Yuan, Y. Zhu, S. Qi, and S.-C. Zhu, "Holistic++ scene understanding: Single-view 3D holistic scene parsing and human pose estimation with human-object interaction and physical commonsense," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 8647–8656.

[7] S. Liu and C.-S. Bouganis, "Communication-aware MCMC method for big data applications on FPGAs," in *Proc. IEEE 25th Annu. Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, Apr. 2017, pp. 9–16.

[8] J. Lao et al., "tfp.Mcmc: Modern Markov chain Monte Carlo tools built for modern hardware," 2020, *arXiv:2002.01184*.

[9] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnol.*, vol. 15, no. 7, pp. 529–544, Jul. 2020.

[10] R. Cai et al., "VIBNN: Hardware acceleration of Bayesian neural networks," in *Proc. Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2018, pp. 476–488.

[11] S. S. Banerjee, Z. T. Kalbarczyk, and R. K. Iyer, "AcMC^2: Accelerating Markov chain Monte Carlo algorithms for probabilistic models," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 515–528.

[12] P. Shukla, A. Shylendra, T. Tulabandhula, and A. R. Trivedi, "MC^{2}RAM: Markov chain Monte Carlo sampling in SRAM for fast Bayesian inference," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.

[13] T. Dalgaty, N. Castellani, C. Turck, K.-E. Harabi, D. Querlioz, and E. Vianello, "In situ learning using intrinsic memristor variability via Markov chain Monte Carlo sampling," *Nature Electron.*, vol. 4, no. 2, pp. 151–161, Jan. 2021.

[14] I. Yildirim, "Bayesian inference: Metropolis–Hastings sampling," Dept. Brain Cogn. Sci., Univ. Rochester, Rochester, NY, USA, Tech. Rep., 2012. [Online]. Available: https://nlp.jbnu.ac.kr/PGM/slides_other/MetropolisHastingsSampling.pdf

[15] M. Plummer, N. Best, K. Cowles, and K. Vines, "CODA: Convergence diagnosis and output analysis for MCMC," *R News*, vol. 6, no. 1, pp. 7–11, 2006.

[16] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[17] S. Yu, "Neuro-inspired computing with emerging nonvolatile memorys," *Proc. IEEE*, vol. 106, no. 2, pp. 260–285, Feb. 2018.

[18] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28 nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 188–190.

[19] T. Moon et al., "Parallel synaptic design of ferroelectric tunnel junctions for neuromorphic computing," *Neuromorphic Comput. Eng.*, vol. 3, no. 2, Jun. 2023, Art. no. 024001.

[20] B. Pan, G. Wang, H. Zhang, W. Kang, and W. Zhao, "A mini tutorial of processing in memory: From principles, devices to prototypes," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 7, pp. 3044–3050, Jul. 2022.

[21] X. Si et al., "A 28 nm 64 Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 246–248.

[22] W.-S. Khwa et al., "A 65 nm 4 Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8 TOPS/W fully parallel product-sum operation for binary DNN edge processors," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 496–498.

[23] J.-W. Su et al., "A 28 nm 64 Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 240–242.

[24] Y.-D. Chih et al., "An 89 TOPS/W and 16.3 TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.

[25] H. Kim, Q. Chen, T. Yoo, T. T. Kim, and B. Kim, "A 1–16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation," in *Proc. IEEE 45th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2019, pp. 345–348.

[26] J.-W. Su et al., "A 28 nm 384 kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 250–252.

[27] M. E. Sinangil et al., "A 7-nm compute-in-memory SRAM macro supporting multi-bit input, weight and output and achieving 351 TOPS/W and 372.4 GOPS," *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 188–198, Jan. 2021.

[28] H. Zhang et al., "CP-SRAM: Charge-pulsation SRAM marco for ultra-high energy-efficiency computing-in-memory," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 109–114.

[29] N. H. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*. London, U.K.: Pearson, 2015.

[30] B. Yan et al., "Resistive memory-based in-memory computing: From device and large-scale integration system perspectives," *Adv. Intell. Syst.*, vol. 1, no. 7, Nov. 2019, Art. no. 1900068.

[31] H. Qin, Y. Cao, D. Markovic, A. Vladimirescu, and J. Rabaey, "SRAM leakage suppression by minimizing standby supply voltage," in *Proc. SCS. Int. Symp. Signals, Circuits Systems.*, Mar. 2004, pp. 55–60.

[32] B. H. Calhoun and A. P. Chandrakasan, "Static noise margin variation for sub-threshold SRAM in 65-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 41, no. 7, pp. 1673–1679, Jul. 2006.

[33] Y. Kagiyama et al., "Bit error rate estimation in SRAM considering temperature fluctuation," in *Proc. 13th Int. Symp. Quality Electron. Design (ISQED)*, Mar. 2012, pp. 516–519.

[34] M. Yamaoka, C. Yoshimura, M. Hayashi, T. Okuyama, H. Aoki, and H. Mizuno, "A 20k-spin Ising chip to solve combinatorial optimization problems with CMOS annealing," *IEEE J. Solid-State Circuits*, vol. 51, no. 1, pp. 303–309, Jan. 2016.

[35] C. L. Phillips, J. A. Anderson, and S. C. Glotzer, "Pseudo-random number generation for Brownian dynamics and dissipative particle dynamics simulations on GPU devices," *J. Comput. Phys.*, vol. 230, no. 19, pp. 7191–7201, Aug. 2011.

[36] A. A. Rezk, A. H. Madian, A. G. Radwan, and A. M. Soliman, "Reconfigurable chaotic pseudo random number generator based on FPGA," *AEU Int. J. Electron. Commun.*, vol. 98, pp. 174–180, Jan. 2019.

[37] S. Soliman, F. Yuan, and K. Raahemifar, "An overview of design techniques for CMOS phase detectors," in *Proc. IEEE Int. Symp. Circuits Systems.*, vol. 5, May 2002, pp. 457–460.

[38] A. E. Raftery and S. M. Lewis, "Implementing MCMC," in *Markov Chain Monte Carlo in Practice*. Boca Raton, FL, USA: CRC Press, 1996, pp. 115–130.

[39] V. B. Suresh and S. Kundu, "On analyzing and mitigating SRAM BER due to random thermal noise," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Aug. 2013, pp. 159–164.

[40] D. A. Reynolds, "Gaussian mixture models," in *Encyclopedia of Biometrics*, vol. 741, nos. 659–663. Boston, MA, USA: Springer, 2009.

[41] C. B. Do, "The multivariate Gaussian distribution," *Sect. Notes, Lect. Mach. Learn.*, vol. 229, pp. 1–11, 2008. [Online]. Available: https://www.alittlefrog.com/wp-content/uploads/2023/09/Stanford-Gaussian-Distribution.pdf

[42] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*. Amsterdam, The Netherlands: Elsevier, 2006.

[43] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[44] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[45] J. Bradbury et al., *JAX: Autograd and XLA*, document ascl-2111, Astrophysics Source Code Library, 2021.



Yihan Fu received the B.S. degree from Peking University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree with the Institute of Artificial Intelligence and the School of Integrated Circuits. His research interests include probabilistic artificial intelligence processors and SRAM-based compute-in-memory designs.



Daijing Shi received the B.S. degree from Peking University, Beijing, China, in 2022, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His research interests include designing RRAM-based in-memory computing circuits and exploring IMC architecture adapting to different artificial intelligent applications and researching reliable multi-level computing methods.



Anjunyi Fan (Student Member, IEEE) received the B.S. degree from Peking University, Beijing, China, in 2022, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His research interests include SRAM-based processing-in-memory designs and their emerging computing patterns.



Wenshuo Yue received the B.S. degree from Peking University, Beijing, China, in 2021, where he is currently pursuing the Ph.D. degree with the School of Integrated Circuits. His research interests include the integration of RRAM and RRAM-based computing systems.



Yuchao Yang (Member, IEEE) received the Ph.D. degree from Tsinghua University in 2010. After that, he joined the University of Michigan, Ann Arbor, as a Post-Doctoral Research Fellow and was promoted to a Senior Research Fellow, in 2013. He joined Peking University, as an Assistant Professor, in 2015, where he is currently a Professor with tenure with the School of Integrated Circuits. He was the Director of the Center for Brain Inspired Chips. He has published over 120 papers and five book chapters. His research interests include memristors, neuromorphic computing, and in-memory computing chips. He is an editorial board member of five journals. He was recognized as a Highly Cited Chinese Researcher by Elsevier in 2020 and 2021. He was a recipient of the Qiu Shi Outstanding Young Scholar Award, the Wiley Young Researcher Award, the MIT Technology Review Innovators Under 35 in China, and the Explorer Prize. He was invited to give over 40 keynote/invited talks at international conferences and serves as the TPC chair or a member for nine international conferences. He serves as an Associate Editor for *Microelectronic Engineering* and an Associate Editor for *Nano Select*.



Ru Huang (Fellow, IEEE) received the B.S. (Hons.) and M.S. degrees in electronic engineering from Southeast University, Nanjing, China, in 1991 and 1994, respectively, and the Ph.D. degree in microelectronics from Peking University, Beijing, China, in 1997. Since 1997, she has been a Faculty Member with Peking University, where she is currently the Boya Chair Professor. She is an elected Academician with the Chinese Academy of Sciences. She has authored or coauthored five books, five book chapters, and more than 300 papers, including more than 100 papers in IEDM, VLSI Technology Symposium, IEEE ELECTRON DEVICE LETTERS, and IEEE TRANSACTIONS ON ELECTRON DEVICES. She has delivered over 50 keynote/invited talks at conferences and seminars. She has been granted over 300 patents, including 49 U.S. patents. Her research interests include nano-scaled CMOS devices, ultra-low-power new devices for neuromorphic computing, emerging memory technology, and device variability/reliability.



Bonan Yan (Member, IEEE) received the B.S. degree from Beihang University, Beijing, China, in 2014, the M.S. degree from the University of Pittsburgh, Pittsburgh, PA, USA, in 2017, and the Ph.D. degree from Duke University, Durham, NC, USA, in 2020. He is currently an Assistant Professor with the Institute for Artificial Intelligence, Peking University, Beijing. He is a Peking University Boya Young Fellow. He has published about 50 technical publications and contributed to compute-in-memory VLSI design with digital SRAM-based circuit structures and analogue spike-based memristor-based computing schemes. His research interests include memory-centric processor architectures and artificial general intelligence computing hardware design.